

FlexBIT local adapter guide

Installation configuration and telemetry testing

This guide is for operators installing the released FlexBIT local adapter. It covers Docker Compose and standalone Linux installations, FlexBIT credentials, device mappings, HTTP and MQTT tests, and routine operations. Commands use the installed deployment files, not the repository's development Compose file.

The adapter accepts device telemetry, stores it in SQLite, maps it to FlexBIT fields, and publishes it to the configured FlexBIT agent. A successful intake test is the first check. Confirm the event's publication status and its arrival in FlexBIT before treating the deployment as complete.

Choose an installation mode

- Docker runs the adapter as a service and can start a Mosquitto broker, QuestDB history, Prometheus, and Grafana through optional profiles.
- Standalone runs only the adapter with bundled Node 24. MQTT and history can use services that you manage separately. No Docker or host Node installation is needed on the target machine.

Before you start

Obtain the FlexBIT site ID, API ID, API secret, and agent endpoint for your environment. You also need the FlexBIT asset IDs that your local device IDs will represent.

Docker requires Docker Engine and the Compose plugin. Standalone requires 64-bit Linux on AMD64 or ARM64, glibc 2.28 or newer, and libstdc++ 6.0.25 or newer. Use a 64-bit OS on Raspberry Pi. Alpine Linux, 32-bit Linux, macOS, and Windows are not standalone targets.

Downloads need access to GitHub, plus ghcr.io for Docker. Standalone downloads require curl, unzip, and sha256sum. REST tests need curl with --fail-with-body support. MQTT tests need mosquitto_pub from the Mosquitto clients.

Guide contents

- Docker installation and startup
- Standalone installation and startup
- FlexBIT configuration and device mappings
- MQTT configuration for both modes
- Telemetry testing and verification
- Operations and troubleshooting

The test helper described here is included in standalone packages built from the revision that adds it. Older ZIPs may contain only start-adapter.sh; install a release that includes test-telemetry.sh to use the standalone helper.

Docker installation and startup

Run these commands on the deployment host. Download the installer to a working directory, then choose an installation directory separate from any standalone installation.

```
curl -fsSL -o install.sh \
  https://raw.githubusercontent.com/CETP-FlexBIT/local-adapter/main/install.sh
sh install.sh --with-docker \
  --install-dir "$HOME/flexbit-local-adapter" --no-start
cd "$HOME/flexbit-local-adapter"
```

In an interactive terminal, choose testing or production and enter the site ID, API ID, and API secret. Existing .env or exported values become defaults. The installer requires all three credentials and writes .env with mode 0600.

For an unattended installation, export FLEXBIT_SITE_ID, FLEXBIT_API_ID, and FLEXBIT_API_SECRET first. Add --environment testing or --environment production. Use --version X.Y.Z to choose a specific release. Keep secrets out of shared command histories.

Start the adapter

```
docker compose up -d
docker compose logs -f adapter
```

Open <http://127.0.0.1:4000> on the host. The installed Compose file uses container port 4000. Its default host mapping does not restrict the bind address; set ADAPTER_PORT=127.0.0.1:4000 in .env to bind HTTP to loopback only. A remote browser's 127.0.0.1 refers to that browser's own machine.

Add optional services

Select services during installation with --with-mqtt, --with-history, and --with-monitoring. For example, install with --with-mqtt before running the following so the adapter subscriptions are configured too.

```
docker compose --profile mqtt --profile history \
  --profile monitoring up -d
```

Repeat the required profile flags when creating or recreating optional services. Starting a broker profile alone does not create adapter subscriptions. The history installer option sets QUESTDB_HTTP_URL=<http://questdb:9000>.

Default optional service addresses are MQTT on port 1883, QuestDB at <http://127.0.0.1:9000>, Prometheus at <http://127.0.0.1:9090>, and Grafana at <http://127.0.0.1:3001>. Set a Grafana admin password in .env before first startup.

Stop or apply changes

Use docker compose stop adapter to stop the adapter. After editing .env, use docker compose up -d --force-recreate adapter. After manually editing mapping or MQTT YAML, use docker compose restart adapter.

Standalone installation and startup

Run this on a supported Linux host. The installer downloads the matching architecture ZIP, checks it against the release SHA256SUMS, and installs the bundled application and Node runtime.

```
curl -fsSL -o install.sh \
  https://raw.githubusercontent.com/CETP-FlexBIT/local-adapter/main/install.sh
sh install.sh --without-docker \
  --install-dir "$HOME/flexbit-local-adapter" --no-start
cd "$HOME/flexbit-local-adapter"
HOST=127.0.0.1 ./start-adapter.sh
```

Provide the credentials when prompted. For unattended installation, export FLEXBIT_SITE_ID, FLEXBIT_API_ID, and FLEXBIT_API_SECRET first and choose --environment testing or production. The explicit HOST value above keeps this startup bound to loopback, including launchers that export their own HOST default.

The adapter runs in the foreground. Leave this terminal open and use a second terminal for testing. Ctrl-C stops the adapter. Open <http://127.0.0.1:4000> on the deployment host.

Use an extracted release ZIP

Download local-adapter-X.Y.Z-linux-amd64.zip or local-adapter-X.Y.Z-linux-arm64.zip and the matching SHA256SUMS from the same GitHub release. Verify the ZIP's checksum before extracting it. In the extracted directory, run its install.sh --without-docker, optionally with --install-dir to copy into a separate installation directory. No dependency installation or build is required.

Installed files

- .output contains the application; runtime contains Node and its license.
- start-adapter.sh starts the server. test-telemetry.sh submits demo telemetry.
- .env contains connection settings and paths; config contains YAML configuration.
- data/adapter/adapter.db stores intake, event state, and settings saved in the UI.

The installer creates config/mqtt.yaml as an empty list. This is valid for REST-only use. --with-mqtt, --with-history, and --with-monitoring are Docker-only installer options and are rejected in standalone mode.

Change the port or run on boot

Set PORT=5050 in .env to change the standalone HTTP port, then restart the process. Export HOST explicitly when starting if you need to control the bind address. Update ADAPTER_URL on test commands to match the new port.

For startup on boot, create a systemd service with the installation owner as User, an absolute ExecStart pointing to start-adapter.sh, Environment=HOST=127.0.0.1, and Restart=on-failure. Enable and start it with systemctl, and read its logs with journalctl -u your-service-name. Stop it before reinstalling.

FlexBIT configuration and device mappings

Open the Settings page at <http://127.0.0.1:4000/settings>. Review the site ID, client ID, agent endpoint, API ID, and API secret. The publisher requires an endpoint, API ID, and API secret. The site ID selects the FlexBIT destination topic.

Settings saved through the UI live in SQLite and override .env defaults. An empty secret submission keeps the saved secret. Reset settings in the UI to restore environment values. After changing .env, recreate the Docker container or restart the standalone process.

Connection defaults

```
FLEXBIT_SITE_ID=your-site-id
FLEXBIT_CLIENT_ID=flexbit-adapter
FLEXBIT_AGENT_ENDPOINT=your-agent-host:9092
FLEXBIT_API_ID=your-api-id
FLEXBIT_API_SECRET=your-api-secret
```

Choose the environment in the installer to populate its configured agent endpoint. Confirm the endpoint with your FlexBIT operator if your deployment uses another agent. The agent setting is a Kafka bootstrap host and port, not an HTTPS URL.

Map your device IDs

The demo helper sends device.id as demo-bess, demo-ev-charger, or demo-pv by default. Replace the shipped demo asset associations with assets belonging to your own site, using the UI or config/device-mappings.yaml.

```
schema_version: 1
device_ids:
  demo-bess: your-bess-flexbit-asset-id
  demo-ev-charger: your-ev-flexbit-asset-id
  demo-pv: your-pv-flexbit-asset-id
```

Merge these entries into your existing file if it contains other devices. Without a matching entry, the mapper keeps the external device ID unchanged. The shipped EV charger demo has no device association, so configure it before checking publication to a real asset.

Configure payload mappings

The installed config/mappings directory includes demo-bess, demo-ev-charger, and demo-pv. Each mapping defines asset_type, asset_id_path, timestamp handling, and fields. For REST, X-Mapping-Id chooses a mapping. For MQTT, each subscription chooses its mappingId.

Use the Mappings page to adjust source paths and target FlexBIT fields. The supplied samples match the supplied demo mappings. A device with another payload shape needs its own mapping. UI saves reload the affected configuration; restart after manual YAML edits.

Protect .env, configuration files containing broker passwords, and SQLite backups. HTTP intake currently does not authenticate requests. The helper's bearer header does not enforce access control, so choose the HTTP bind address and host access policy deliberately.

MQTT configuration for both modes

MQTT needs two separate pieces: a reachable broker and an adapter subscription. Installing the Mosquitto clients gives you `mosquitto_pub`; it does not start a broker or configure the adapter.

Docker broker

Install with `--with-mqtt` to configure the local subscriptions. The adapter connects to `mqtt://mosquitto:1883` inside the Compose network. A helper run on the host connects to `127.0.0.1:1883`. Use `MQTT_BIND_ADDRESS=127.0.0.1` in `.env` for local testing, then start the broker profile.

```
docker compose --profile mqtt up -d
./scripts/test-telemetry.sh mqtt bess
```

Standalone broker

Start a broker you manage separately. Open the adapter MQTT page or edit `config/mqtt.yaml`. This example connects to a broker on the same host and subscribes to all demo topics.

```
- id: local
  url: mqtt://127.0.0.1:1883
  clientId: flexbit-local-adapter
  subscriptions:
    - topic: devices/bess/telemetry
      mappingId: demo-bess
      qos: 1
      retained: ignore
    - topic: devices/ev-charger/telemetry
      mappingId: demo-ev-charger
      qos: 1
      retained: ignore
    - topic: devices/pv/telemetry
      mappingId: demo-pv
      qos: 1
      retained: ignore
```

Restart the adapter after a manual edit. In another terminal, run `./test-telemetry.sh mqtt bess` from the installation directory. If the broker is remote, use its host in both the adapter URL and the helper's `MQTT_HOST` override.

Credentials and delivery checks

For an authenticated broker, add username and password to the connection entry, then set `MQTT_USERNAME` and `MQTT_PASSWORD` for the helper. Configure TLS through the adapter's supported `mqttps` URL and use a client command with appropriate TLS options when testing; the helper exposes only host, port, username, and password options.

The helper uses MQTT v5, QoS 1, and non-retained messages. Check connection status on the MQTT page and look for the event on the Events page. A broker acknowledgement alone does not prove that the adapter subscription received or mapped the payload.

Telemetry testing and verification

Start the adapter first. Run the following checks from a second terminal on the deployment host. Use your configured HTTP port instead of 4000 if it differs.

```
curl -fsS http://127.0.0.1:4000/healthz
curl -fsS http://127.0.0.1:4000/readyz
curl -fsS http://127.0.0.1:4000/v1/status
```

Inspect the JSON, especially `mapping_error` and `publisher_enabled`. The readiness endpoint reports state in its response body; a successful curl exit alone does not prove the mapper or publisher is working.

Send REST samples

```
# Docker installation
./scripts/test-telemetry.sh rest bess
./scripts/test-telemetry.sh rest ev-charger
./scripts/test-telemetry.sh rest pv

# Standalone installation
./test-telemetry.sh rest bess
./test-telemetry.sh rest ev-charger
./test-telemetry.sh rest pv
```

Each request uses `POST /v1/ingest` and the corresponding demo mapping ID. Expect HTTP 202 and JSON containing `event_id` and `status accepted`. Save the event ID from the output, then inspect its trace.

```
curl -fsS http://127.0.0.1:4000/v1/events/YOUR_EVENT_ID
```

Open the Events page to inspect mapped fields, publication state, and errors. Confirm the resulting measurement on the correct asset in FlexBIT. This is the end-to-end check; local acceptance can succeed while mapping or publishing fails.

Override the destination or device

```
ADAPTER_URL=http://127.0.0.1:5050 DEVICE_ID=inverter-01 \
./test-telemetry.sh rest pv
MQTT_HOST=broker.example MQTT_PORT=1883 \
./test-telemetry.sh mqtt pv
./test-telemetry.sh all bess
```

For Docker, replace `./test-telemetry.sh` with `./scripts/test-telemetry.sh`. `DEVICE_ID` must match the key in your device mappings. The helper does not read `.env` automatically. Export `ADAPTER_URL`, `MQTT_HOST`, `MQTT_PORT`, and broker credentials explicitly when defaults differ.

With no arguments the helper tests both REST and MQTT for BESS. Use `rest` explicitly for REST-only installations. `all` sends a sample through each transport and can produce two events. `INGESTION_TOKEN`, or `FLEXBIT_INGESTION_TOKEN`, changes the optional bearer header; its fallback is `dev-ingest`.

Operations and troubleshooting

Keep configuration and data

Docker deployments use bind-mounted data/adapter, data/questdb, and data/grafana directories. Standalone uses data/adapter. Back up .env, config, and data together while the adapter and any history services are stopped.

```
# Docker with all optional services
# Omit profiles that you do not use.
docker compose --profile mqtt --profile history \
  --profile monitoring stop
tar -czf /secure/path/flexbit-backup.tar.gz .env config data
docker compose --profile mqtt --profile history \
  --profile monitoring up -d
```

For standalone, stop the foreground process or systemd service, create the same archive from the installation directory, then restart. Store backups securely. The existing backup.sh, restore.sh, and reset.sh helpers target the former Docker named-volume layout and do not manage the installed bind-mounted data directories.

To restore, stop the services, preserve a copy of the current files, and extract the backup into the installation directory. Restart and repeat the health and telemetry checks. Reinstallation preserves .env, operator configuration, and SQLite data; it refreshes the application, runtime, and managed scripts. Stop standalone before reinstalling.

Diagnose a failed test

- Connection refused on REST: start the adapter, inspect logs, and check PORT for standalone or ADAPTER_PORT for Docker. Match ADAPTER_URL to the reachable address.
- Missing curl or mosquitto_pub: install the required client on the machine running the helper. REST-only tests do not need MQTT clients.
- MQTT publish fails: check broker availability, host, port, and credentials. Docker uses mosquitto inside its network; a host helper uses the exposed host address. Standalone needs a separately managed broker.
- MQTT acknowledged but no event: check the adapter connection and topic subscriptions. Match mappingId to an installed mapping. Inspect adapter logs and the MQTT page.
- Accepted event but mapping fails: inspect mapping_error and the event trace. Verify source paths, field types, timestamp format, and device IDs. Restart after manual YAML changes.
- Mapped event remains pending: review the endpoint, site ID, API credentials, network access, and asset association. Check for saved UI settings overriding .env.
- HTTP 429: intake capacity is exceeded. Inspect pending events, publisher failures, and disk capacity. Observe the Retry-After response instead of repeatedly sending more samples.
- Bundled Node cannot run: verify Linux architecture, glibc, and libstdc++. Use the matching 64-bit ZIP.

Stop or remove the installation

Docker installations include scripts/uninstall.sh to stop installed profiles without deleting bind-mounted data. In standalone, stop the process or service. Removing the installation directory deletes its local configuration and data, so take a backup first.

For configuration details and release downloads, use the project documentation at <https://cetp-flexbit.github.io/local-adapter/> and <https://github.com/CETP-FlexBIT/local-adapter/releases>. The guide's commands follow install.sh, compose.release.yaml, the bundled scripts, and the adapter's intake and configuration code.